

Attempt 1



In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

▼ Details

COE 311K Final Project - Part 2: Stiff ODEs and Implicit Methods

Overview

In Part 2 of the final project, you will solve a **stiff** ordinary differential equation (ODE) that arises in real-world applications. Stiff equations require special numerical treatment because explicit methods (like Euler's Forward and RK4 from Part 1) require impractically small step sizes to remain stable. You will implement **Euler's Backward (Implicit Euler) method** using **Newton-Raphson iteration** to solve the resulting nonlinear equation at each time step.

Learning Objectives

By completing this project, you will:

- Understand the concept of stiffness in ODEs and why it matters
- Implement an implicit numerical method for stiff problems
- Apply Newton-Raphson iteration to solve nonlinear algebraic equations
- Compare implicit vs. explicit method stability for stiff problems
- Gain experience with challenging real-world engineering problems

What are Stiff ODEs?

A system of ODEs is considered **stiff** when:

- It contains processes occurring on vastly different time scales
- Explicit methods require extremely small step sizes for stability (even when accuracy doesn't demand it)
- Implicit methods can use much larger step sizes while remaining stable

Key characteristic: The stability requirement is much more restrictive than the accuracy requirement.

Project Requirements

1. Stiff Scalar ODE Selection

Choose **ONE** of the following stiff scalar ODE applications:

Option A: RC Circuit with Fast Transient

A simple RC circuit charging/discharging with very small time constant:

$$\frac{dV}{dt} = -\frac{1}{RC}V + \frac{1}{RC}V_{\text{source}}(t)$$

For stiffness, use:

- $R = 100\Omega$
- $C = 10^{-9}\text{F}$ (1 nanofarad)
- $RC = 10^{-7}$ seconds (very fast!)
- $V_{\text{source}}(t) = 5 \sin(\omega t)\text{V}$, with $\omega = 1$ rad/s (slow input)

Why it's stiff: The circuit responds in 0.1 microseconds, but the input signal varies over seconds. Time scale ratio: 10^7 .

Rewrite in standard form:

$$\frac{dV}{dt} = -10^7 V + 10^7 \cdot 5 \sin(t)$$

Applications: Digital circuits, high-speed electronics, sensor signal conditioning

Option B: Chemical Reaction with Fast Kinetics

A simple first-order chemical decay approaching equilibrium:

$$\frac{dC}{dt} = -k(C - C_{\text{eq}})$$

For stiffness, use:

- $k = 10^4 \text{s}^{-1}$ (very fast reaction rate)

- $C_{\text{eq}} = 1 + 0.1 \sin(0.1t)$ (slowly varying equilibrium)
- Initial condition: $C(0) = 0$

Rewrite in standard form:

$$\frac{dC}{dt} = -10^4 C + 10^4 (1 + 0.1 \sin(0.1t))$$

Why it's stiff: Reaction occurs in milliseconds, but equilibrium varies over tens of seconds. Ratio: 10^5 .

Applications: Fast enzyme kinetics, combustion chemistry, atmospheric reactions

Option C: Cooling with Forced Convection

Newton's law of cooling for a hot object in fluid flow:

$$\frac{dT}{dt} = -h(T - T_{\infty}(t))$$

For stiffness, use:

- $h = 1000\text{s}^{-1}$ (very high convection coefficient)
- $T_{\infty}(t) = 300 + 50 \sin(0.01t)\text{K}$ (slowly varying ambient)
- Initial condition: $T(0) = 500\text{K}$

Rewrite in standard form:

$$\frac{dT}{dt} = -1000T + 1000(300 + 50 \sin(0.01t))$$

Why it's stiff: Object cools in milliseconds, but ambient temperature varies over minutes. Ratio: 10^5 .

Applications: Heat exchangers, cooling systems, thermal management

Option D: Mechanical System with High Damping

Overdamped spring-mass system (converted to first-order by assuming $m \rightarrow 0$):

$$\frac{dx}{dt} = -\frac{k}{c}x + \frac{1}{c}F(t)$$

For stiffness, use:

- $k/c = 5000\text{s}^{-1}$ (very high damping rate)
- $F(t) = 10 \sin(t)\text{N}$ (slow external force)
- Initial condition: $x(0) = 1\text{m}$

Rewrite in standard form:

$$\frac{dx}{dt} = -5000x + 0.002 \sin(t)$$

Why it's stiff: System responds in ~ 0.2 milliseconds, but force varies over seconds. Ratio: 5×10^3 .

Applications: Shock absorbers, vibration dampers, seismic isolation

Option E: Population with Rapid Birth/Death Cycles

Simplified population model with fast equilibration:

$$\frac{dN}{dt} = -\lambda(N - N_{\text{carrying}}(t))$$

For stiffness, use:

- $\lambda = 10^3$ (rapid population turnover, think bacteria)
- $N_{\text{carrying}}(t) = 10^6(1 + 0.5 \sin(0.1t))$ (slowly varying resources)
- Initial condition: $N(0) = 10^5$

Rewrite in standard form:

$$\frac{dN}{dt} = -10^3 N + 10^3 \cdot 10^6(1 + 0.5 \sin(0.1t))$$

Why it's stiff: Population equilibrates in seconds, but resources vary over tens of seconds. Ratio: $10^2 - 10^3$.

Applications: Bacterial cultures, rapid reproduction species, viral dynamics

Common Form for All Problems

Notice all these problems have the form:

$$\frac{dy}{dt} = -\lambda y + g(t)$$

where:

- $\lambda > 0$ is **large** (this creates stiffness)
- $g(t)$ is a **slowly varying** forcing function

Analytical solution exists:

$$y(t) = e^{-\lambda t} y_0 + \int_0^t e^{-\lambda(t-s)} g(s) ds$$

This will help you verify your numerical solution!

2. Demonstrating Stiffness

Before implementing Euler's Backward, you must **demonstrate that your problem is stiff**:

Required Analysis:

1. **Attempt Euler's Forward** with reasonable step size (e.g., $h = 0.1$)
 - Show that it becomes unstable or requires tiny step sizes
 2. **Calculate or estimate the stiffness ratio**:
 - For linear systems: ratio of largest to smallest eigenvalue magnitude (challenge 17b)
 - For nonlinear: ratio of fastest to slowest time scales
 - Document why your system is stiff
 3. **Show the impracticality** of explicit methods:
 - Estimate the step size required for Euler's Forward stability
 - Calculate how many steps would be needed for the full simulation
 - Demonstrate computational burden
-

3. Euler's Backward Method with Newton-Raphson

The Implicit Euler Method

For the scalar ODE $\frac{dy}{dt} = f(t, y)$, Euler's Backward is:

$$y_{n+1} = y_n + h \cdot f(t_{n+1}, y_{n+1})$$

This is **implicit** because y_{n+1} appears on both sides. For nonlinear f , this requires solving a nonlinear equation at each step.

Newton-Raphson Iteration

Define the residual function:

$$g(y_{n+1}) = y_{n+1} - y_n - h \cdot f(t_{n+1}, y_{n+1}) = 0$$

Newton-Raphson iteration to solve $g(y_{n+1}) = 0$:

$$y_{n+1}^{(k+1)} = y_{n+1}^{(k)} - \frac{g(y_{n+1}^{(k)})}{g'(y_{n+1}^{(k)})}$$

Where the derivative is:

$$g'(y_{n+1}) = 1 - h \cdot \left. \frac{\partial f}{\partial y} \right|_{t_{n+1}, y_{n+1}}$$

Note: For the standard form $\frac{dy}{dt} = -\lambda y + g(t)$:

- $f(t, y) = -\lambda y + g(t)$
- $\frac{\partial f}{\partial y} = -\lambda$
- $g'(y_{n+1}) = 1 - h \cdot (-\lambda) = 1 + h\lambda$

This is particularly simple! The derivative g' is constant, so you compute it once.

Implementation Requirements:

1. **Derive $g'(y_{n+1})$** for your specific problem
 2. **Implement Newton-Raphson solver:**
 - Initial guess: $y_{n+1}^{(0)} = y_n$ (explicit Euler predictor)
 - Convergence criterion: $|g(y_{n+1}^{(k)})| < \text{tol}$ (e.g., $\text{tol} = 10^{-6}$)
 - Maximum iterations: typically 10-20
 - Track number of iterations per time step
 3. **Implement Euler's Backward:**
 - At each time step, call Newton-Raphson to solve for y_{n+1}
 - Use multiple step sizes to analyze performance
 - Monitor convergence behavior
-

4. Stability Analysis and Comparison

Your analysis must include:

a) Step Size Comparison

- Compare the **maximum stable step size** for:
 - Euler's Forward (from demonstration of stiffness)
 - Euler's Backward (show it can handle much larger steps)
- Create a table showing stable step size ranges

b) Accuracy vs. Computational Cost

- For Euler's Backward with different step sizes:
 - Total number of time steps
 - Average Newton-Raphson iterations per step
 - Total function evaluations
- Compare to the cost of Euler's Forward with stable step size

c) Convergence Behavior

- Plot Newton-Raphson iterations vs. time
- Show how convergence depends on:
 - Step size
 - Initial guess quality
 - System nonlinearity

d) Solution Quality

- If analytical or reference solution exists, plot errors
- For stiff systems, verify:
 - Solution remains in physical bounds (concentrations ≥ 0 , etc.)
 - Fast transients are resolved correctly
 - Slow dynamics match expected behavior

Deliverable: Jupyter Notebook

Submit a **single Jupyter Notebook** pdf with the following structure:

Section 1: Introduction & System Selection (~0.5 pages)

- Description of your chosen stiff system
- The governing ODEs
- Physical meaning of variables and parameters
- Why this system is important in practice

Section 2: Demonstration of Stiffness (~0.5 pages + plots)

- Attempt to solve with Euler's Forward
- Show instability or impractical step size requirements
- Calculate/estimate stiffness ratio
- Explain why implicit methods are necessary

Section 3: Mathematical Setup (~0.5 pages)

- Residual function $g(y_{n+1})$ for your specific problem
- **Analytical derivation** of $g'(y_{n+1})$
- Newton-Raphson update formula for your specific problem
- Note: For the linear form $dy/dt = -\lambda y + g(t)$, this is particularly simple!

Section 4: Implementation (~0.25 pages + code)

- Well-commented code for:
 - Function $f(t, y)$ defining your ODE
 - Derivative $g'(y_{n+1})$ (or just use the constant value for linear problems!)
 - Newton-Raphson solver
 - Euler's Backward method with NR iteration

Section 5: Results & Analysis (~0.75 pages + plots)

- Solutions for multiple step sizes
- Comparison plots showing solution behavior
- Newton-Raphson convergence analysis
- Step size stability comparison

Section 6: Performance Analysis (~0.5 pages + table/plots)

- Computational cost analysis
- Table comparing Euler's Forward vs. Backward
- Discussion of accuracy-efficiency tradeoff
- Practical recommendations

Section 7: Conclusions (~0.25 pages)

- Summary of key findings
 - When to use implicit vs. explicit methods
 - Challenges encountered and lessons learned
-

Formatting Guidelines

Length

- **Total length:** 6-8 pages when printed (including code and plots)
- **Text/markdown:** ~2.5-3 pages
- **Code:** ~1.5-2 pages
- **Plots:** ~2-3 pages (5-7 key figures)

Code Requirements

- Separate, modular functions for each component
- Clear variable names matching mathematical notation
- Comments explaining Newton-Raphson convergence checks
- Proper error handling (e.g., max iterations exceeded)

Plot Requirements

All plots must have:

- Descriptive titles
- Labeled axes with units
- Legends when appropriate
- Readable font sizes

Key plots to include:

1. Demonstration of stiffness (Euler's Forward instability)
 2. Solution comparison (different step sizes with Euler's Backward)
 3. Newton-Raphson iterations per time step
 4. Error vs. step size (if reference solution available)
 5. Phase portrait or concentration profiles
-

Grading Rubric (40 points)

System Selection & Stiffness Demonstration (10 points)

- Appropriate stiff problem chosen (2 pts)

- Clear demonstration that system is stiff (4 pts)
- Quantitative stiffness analysis (time scale ratio) (4 pts)

Mathematical Formulation (6 points)

- Correct residual function $g(y_{n+1})$ (3 pts)
- Correct analytical derivative $g'(y_{n+1})$ (3 pts)

Code Implementation (10 points)

- Correct Newton-Raphson solver (4 pts)
- Correct Euler's Backward implementation (4 pts)
- Code quality: comments, modularity, error handling (2 pts)

Analysis & Results (10 points)

- Step size stability analysis (4 pts)
- Convergence behavior analysis (4 pts)
- Computational cost comparison (2 pts)

Presentation (4 points)

- Clear, well-labeled plots (1 pts)
- Well-written explanations (2 pts)
- Overall organization (1 pts)

Submission Instructions

- **Due Date:** April 30th, 2026
- **Submit via Canvas:** Upload your .pdf file
- **File naming:** `LastName_FirstName_COE311K_Part2.pdf`
- **Ensure your notebook runs:** Restart kernel and run all cells before submitting

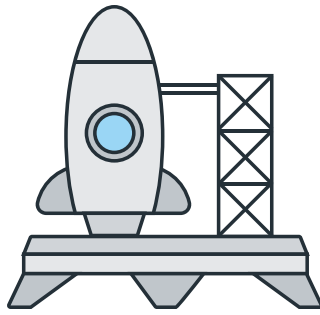
Choose a submission type

Upload

⋮

More





Drag a file here, or

[Choose a file to upload](#)

or



Webcam Photo



Canvas Files

Submit Assignment